

LLM Chat App — Project Report

1. API Structure

The API is built with FastAPI and follows REST conventions. All protected routes require a JWT access token in the Authorization: Bearer <token> header.

Auth Routes

Method

Endpoint

Auth Required

Description

POST

/auth/register

No

Register with username, email, password

POST

/auth/login

No

Login, returns access token + sets refresh cookie

POST

/auth/refresh

No

Uses refresh token cookie to issue new access token

POST

/auth/logout

No

Deletes refresh token from Redis

GET

/auth/github

No

Redirects to GitHub OAuth page

GET

/auth/github/callback

No

GitHub redirects here with code, issues tokens

Chat Routes

Method

Endpoint

Auth Required

Description

POST

/chats/

Yes

Create a new chat thread

GET

/chats/

Yes

Get all chats for current user

GET

/chats/{id}

Yes

Get a single chat with all its messages

PATCH

/chats/{id}

Yes

Update chat title

DELETE

/chats/{id}

Yes

Delete chat and all its messages

Message Routes

Method

Endpoint

Auth Required

Description

GET

/chats/{id}/messages

Yes

Get all messages in a chat

POST

/chats/{id}/messages

Yes

Send message, get full LLM response

POST

/chats/{id}/messages/stream

Yes

Send message, get streaming LLM response

How Clients Use the API

1. Client calls POST /auth/login → receives access_token in response body
2. Client stores access_token in localStorage

3. Every subsequent request includes header: Authorization: Bearer <token>
4. When access token expires (401 response) → client calls POST /auth/refresh → gets new access token
5. For streaming responses, client uses the Fetch API with a ReadableStream reader to read Server-Sent Events token by token

2. Code Structure

llm-chat-app/

```
├─ app/
|   ├── main.py # App entry point, middleware, router registration
|   ├── core/
|   |   ├── __init__.py # Loads all env variables via python-dotenv
|   |   ├── database.py # MongoDB connection (motor async client)
|   |   ├── redis.py # Redis connection (redis.asyncio client)
|   |   └── dependencies.py # get_current_user — JWT auth middleware
|   ├── models/
|   |   ├── user.py # UserInDB, RegisterRequest, LoginRequest, UserResponse
|   |   └── chat.py # Chat, Message, CreateChatRequest, SendMessageRequest
|   ├── controllers/
|   |   ├── auth_controller.py # /auth routes — register, login, refresh, logout, github
|   |   └── chat_controller.py # /chats routes — CRUD + message sending + streaming
|   └── services/
|       ├── auth_service.py # Password hashing, JWT creation, refresh token logic
|       ├── chat_service.py # Chat/message CRUD + Redis caching logic
|       ├── github_service.py # GitHub OAuth token exchange + user fetch
|       └── llm_service.py # Mistral AI API calls (normal + streaming)
```

- └─ frontend/
 - └─ src/
 - └─ api.js # Axios instance with auth interceptors
 - └─ context/
 - └─ AuthContext.jsx # Global auth state, login/logout functions
 - └─ pages/
 - └─ Login.jsx # Login form + GitHub OAuth button
 - └─ Register.jsx # Registration form
 - └─ Chat.jsx # Main chat page with streaming
 - └─ components/
 - └─ Sidebar.jsx # Chat list, new chat button, user info
 - └─ MessageBubble.jsx # Individual message display
 - └─ .env # Environment variables (not committed to git)
 - └─ requirements.txt # Python dependencies
 - └─ README.md # Setup and run instructions

Key Modules Explained

app/main.py — Creates the FastAPI app, registers CORS middleware, registers all routers, and manages startup/shutdown lifecycle using the modern lifespan context manager.

app/core/dependencies.py — The `get_current_user` function acts as auth middleware. Any route that includes `Depends(get_current_user)` is automatically protected. It extracts the Bearer token, decodes the JWT, and fetches the user from MongoDB.

app/services/auth_service.py — Contains all auth logic: bcrypt password hashing/verification, JWT access token creation/decoding, and Redis-based refresh token management (create, verify, delete).

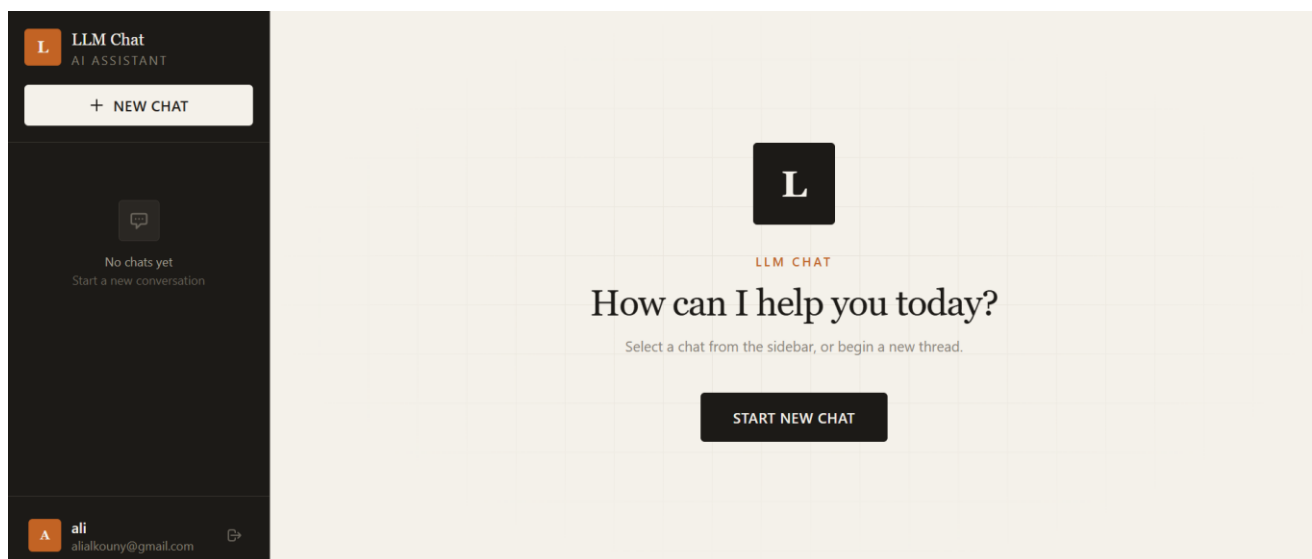
app/services/chat_service.py — Handles all chat and message operations. Includes a Redis caching layer: messages are cached for 30 minutes on first load, cache is invalidated when new messages are added or chat is deleted.

app/services/llm_service.py — Two functions: `get_llm_response()` for standard responses and `stream_llm_response()` which is an async generator that yields tokens one by one using Mistral's streaming API.

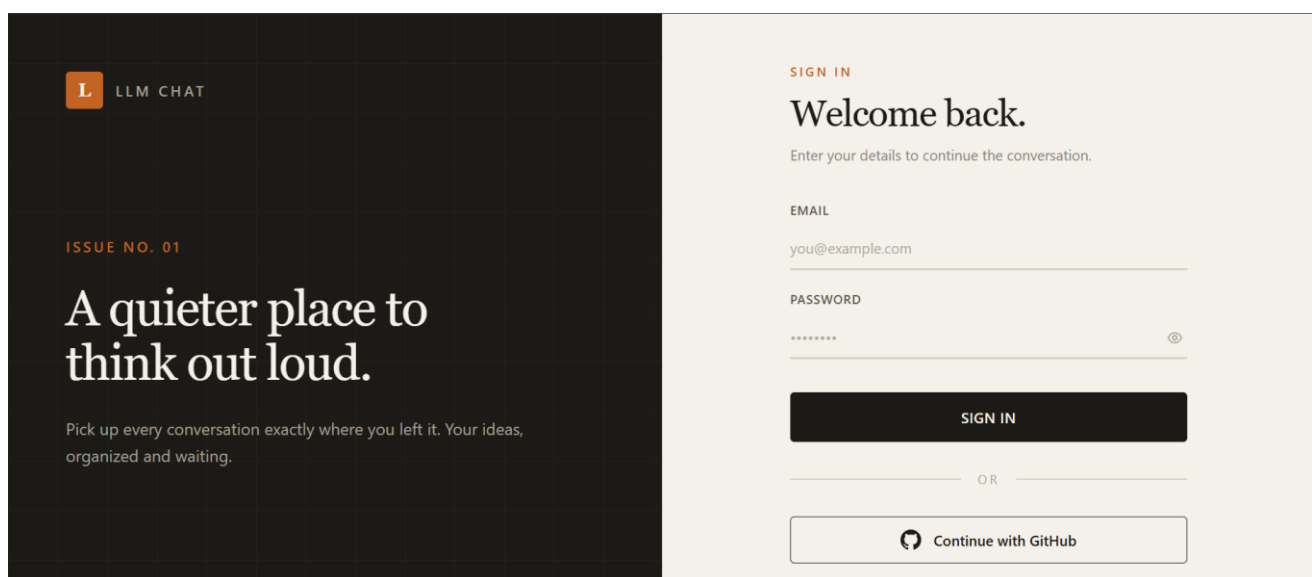
frontend/src/api.js — Axios instance with two interceptors: one that automatically attaches the JWT token to every request, and one that automatically refreshes the token on 401 responses.

3. UI Screenshots

Login Page



Register Page



Chat Page:

L

LLM Chat

AI ASSISTANT

+ NEW CHAT

RECENT

tell me about ITMO university

ali

alialkouny@gmail.com

L

tell me about ITMO university

AI ASSISTANT

ONLINE

tell me about ITMO university

You

AI

ITMO University (Saint Petersburg National Research University of Information Technologies, Mechanics and Optics) is one of Russia's leading universities, specializing in IT, engineering, and applied sciences. Here's an overview of its key aspects:

1. Overview

Established: 1900 (as a technical school), gained university status in 1992.

Location: Saint Petersburg, Russia.

Type: Public research university.

Motto: "Scientia et Ars" (Latin for "Science and Art").

Rankings:

Consistently ranked among the top 10 Russian universities.

Top 300 globally in **Computer Science** (QS World University Rankings).

Top 100 in **Physics & Astronomy** & **Engineering & Technology** (QS)

Message AI Assistant...

Press **Enter** to send - **Shift+Enter** for new line

MongoDB Tables

cloud.mongodb.com/v2/6a09fc122535ffb6f8c35a86#/explorer/6a09fcb05e76bbe3ec0a1cb1/llm_chat

La Liga Gmail Maps StudyApp — олимпиа... What is Subnet Mas... MathDep ITMO Machine Learning S... Experiments - Micro...

ORGANIZATION Ali's Org - 2026-05-17 PROJECT Project 0

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

admin

llm_chat

chats

messages

users

local

sample_mflix

Cluster0 > llm_chat

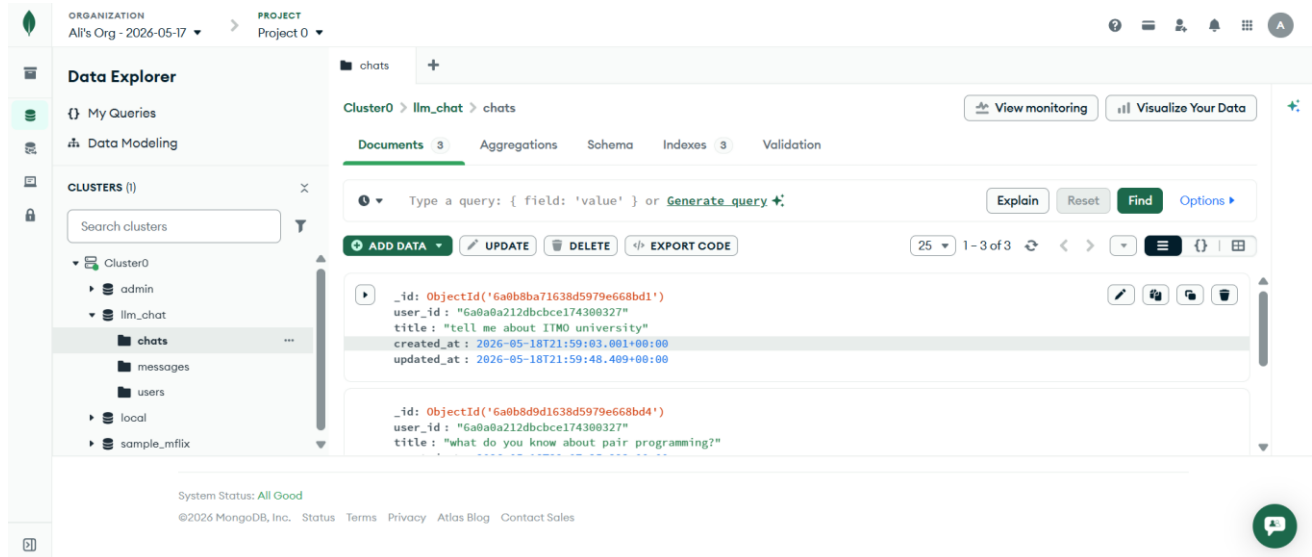
View monitoring Visualize your data Create collection Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
chats	-	36.86 kB	141.00 B	1	141.00 B	3	110.59 kB
messages	-	36.86 kB	3.42 kB	2	1.81 kB	3	110.59 kB
users	-	36.86 kB	185.00 B	1	185.00 B	4	147.46 kB

System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Schema (Chats)



- Login page
- Register page
- Welcome / empty state screen
- Active chat with messages
- Sidebar showing multiple chats
- MongoDB Atlas collections view

4. Database Structure

The database is **MongoDB Atlas** (free M0 cluster). Database name: llm_chat. It contains 3 collections.

Users Collection

Stores all registered users whether they signed up with email/password or GitHub OAuth.

```
{  ObjectId — auto-generated unique ID
  "username": String — unique display name
  "email": String — unique email address
  "hashed_password": String | null — bcrypt hash, null for GitHub-only users
  "github_id": String | null — GitHub user ID, null for email users
}
```


"created_at": DateTime — registration timestamp

}

Chats Collection

Each document represents one conversation thread belonging to a user.

{

"_id": ObjectId — auto-generated unique ID

"user_id": String — references Users._id (ownership check on every query)

"title": String — auto-set from first message (first 40 chars)

"created_at": DateTime — when chat was created

"updated_at": DateTime — updated on every new message (used for sorting)

}

Messages Collection

Each document is one message inside a chat. Role is either "user" or "assistant".

{ "_id": ObjectId — auto-generated unique ID

"chat_id": String — references Chats._id

"role": String — "user" or "assistant"

"content": String — the message text

"created_at": DateTime — message timestamp (used for ordering)

}

Redis Key Structure

Redis is used for two purposes:

Key Pattern	refresh:<token>	Refresh token session storage	chat_messages:<chat_id>
Value	user_id		JSON array
TTL	30 days		30 minutes
Purpose			Chat history cache

5. Architecture & Design Decisions

Why MCS over MVC? Since the frontend is a React SPA, the backend serves only JSON — no HTML templates. MCS (Model-Controller-Service) is the appropriate pattern: controllers handle HTTP concerns, services contain business logic, and models define data shapes.

Why MongoDB? Chat messages and user data are document-shaped and schema-flexible. MongoDB's document model maps naturally to our data without needing joins or migrations.

Why Redis for both sessions and caching? Redis excels at two things we need: fast key-value TTL storage for refresh tokens (security), and low-latency cache for frequently read chat history (performance). Using one tool for both keeps the stack simple.

JWT Flow: Access tokens are short-lived (30 min) and stored in localStorage for easy use in API calls. Refresh tokens are long-lived (30 days), stored in Redis server-side, and sent only via HTTP-only cookies — making them inaccessible to JavaScript and protected against XSS attacks.

Streaming: The /messages/stream endpoint uses FastAPI's StreamingResponse with Server-Sent Events (SSE) format. The frontend uses the Fetch API's ReadableStream to read tokens as they arrive, updating the UI in real time — giving users the familiar ChatGPT-like typing experience.